

kpnn2 release procedure

The whole release, in order. Replace `X.Y.Z` with the release version (for example `0.2.0`) and `rcN` with the release-candidate suffix (`rc1`, `rc2`, ...). Run every command from the repository root.

WARNING

Pushing any tag that starts with `v` publishes to the real PyPI. Never push a release-candidate tag such as `v0.2.0rc1`. Release candidates reach TestPyPI through the dry-run script only.

NOTE

TestPyPI and PyPI never accept the same version twice. Every dry run needs a new `rcN`.

How the automation works

Trigger	Workflow	What it does
Push to <code>main</code>	<code>ci.yml</code>	Ruff, mypy, numpydoc, tests on Python 3.10–3.14, docs build, wheel smoke test
Push of a <code>v*</code> tag	<code>release.yml</code>	Checks the tag against <code>pyproject.toml</code> , builds, publishes to PyPI after your approval
Push of a <code>v*</code> tag	<code>docs.yml</code>	Runs the notebooks, builds the site, deploys <code>X.Y.Z</code> as <code>latest</code>

CI does not run on tags, so the commit you tag must already be green on `main`.

1. One-time setup

- ☐ TestPyPI API token for `kpnn2` (test.pypi.org → Account settings → API tokens)
- ☐ PyPI trusted publisher for `kpnn2`: repository `Thomas-Rauter/kpnn2`, workflow `release.yml`, environment `pypi`
- ☐ GitHub environment `pypi` with you as required reviewer (repository Settings → Environments)
- ☐ `gh` logged in: `gh auth status`
- ☐ The project venv with the `dev` extra: `pip install -e ".[dev]"`

2. Prepare the release candidate

- ☐ `CHANGELOG.md`: the section for `X.Y.Z` is complete, and its heading carries the release date: `## [X.Y.Z] - D. Month YYYY`
- ☐ `CITATION.cff`: `version: "X.Y.Z"` and `date-released: "YYYY-MM-DD"`
- ☐ `RELEASE_NOTES.md` in the repository root: release date, a few highlights, the link `https://github.com/Thomas-Rauter/kpnn2/blob/vX.Y.Z/CHANGELOG.md`, the install command, and the docs link. The file is gitignored, so it exists only on your machine.

- ☐ `pyproject.toml` and `src/kpnn2/__init__.py` both say `X.Y.ZrcN`. The dry run fails if they differ.
- ☐ Every README image points to a PNG that is already on `main`:
`https://raw.githubusercontent.com/Thomas-Rauter/kpnn2/main/docs/figures/<name>.png`.
PyPI shows neither relative paths nor SVGs.
- ☐ `X.Y.ZrcN` is not on TestPyPI yet: <https://test.pypi.org/project/kpnn2/#history>.
- ☐ Everything is committed and pushed. This changes no files; the first line must read `##`
`main...origin/main`, with nothing listed below it:

```
git fetch origin
git status -sb
```

- ☐ CI is green on that commit:

```
gh run list --workflow ci.yml --limit 3
```

3. TestPyPI dry run

1. Activate the venv and put the TestPyPI token into this shell only. `read -rs` keeps the token out of your shell history: paste it (it starts with `pypi-`) and press Enter.

```
source .venv/bin/activate
export TWINE_USERNAME="__token__"
read -rs TWINE_PASSWORD
export TWINE_PASSWORD
```

2. Run the dry run. It runs Ruff, mypy, and the full test suite, builds, uploads to TestPyPI, installs the candidate into a fresh venv, and runs a smoke test there.

```
./dev/dry_run_testpypi_release.sh
```

3. Remove the token from the shell:

```
unset TWINE_USERNAME TWINE_PASSWORD
```

4. Open `https://test.pypi.org/project/kpnn2/X.Y.ZrcN/`. The README renders with every image, and the version, Python versions, license, and links are right.

IMPORTANT

Continue only if the dry run ends with `TestPyPI install smoke test passed`. If anything fails, fix it, commit, push, wait for green CI, raise `rcN` in both version files, and repeat this section.

4. Colab GPU and TPU check

The notebook installs the newest pre-release from TestPyPI without dependencies, so Colab keeps its own CUDA or XLA PyTorch.

1. Open the committed notebook from GitHub. Never upload a local copy:
https://colab.research.google.com/github/Thomas-Rauter/kpnn2/blob/main/tests/manual/kpnn2_colab_gpu_tpu.ipynb#sandboxMode=true
2. **Runtime** → **Change runtime type** → **T4 GPU**, then **Runtime** → **Run all**.
3. **Runtime** → **Change runtime type** → **TPU v2**, then **Runtime** → **Run all** again.

IMPORTANT

Both runs must end with `ALL ACCELERATOR CHECKS PASSED`, and the line `kpnn2: X.Y.ZrcN` must name the candidate you uploaded. A printed `torch.compile` skip on TPU is not a failure. If a check fails, fix it and go back to section 3 with the next `rcN`.

5. Set the final version

1. Set `X.Y.Z`, without `rcN`, in `pyproject.toml` and `src/kpnn2/__init__.py`.
2. Commit and push:

```
git add pyproject.toml src/kpnn2/__init__.py
git commit -m "Release X.Y.Z"
git push origin main
```

3. Wait until CI is green on this commit:

```
gh run list --workflow ci.yml --limit 1
```

6. Tag and publish to PyPI

1. Set the version in this shell. Section 7 reuses it.

```
VERSION=X.Y.Z
```

2. Take the commit that CI tested, not your local `HEAD`. This prints the SHA of the newest green push to `main`, or nothing if that run did not pass:

```
SHA=$(gh run list --workflow ci.yml --limit 20 \
  --json headSha,conclusion,headBranch,event \
  --jq '[.[] | select(.headBranch == "main" and .event == "push")][0]
    | select(.conclusion == "success") | .headSha')
echo "$SHA"
```

3. Check that it is the release commit and carries the version:

```
git log -1 --oneline "$SHA"
git show "$SHA":pyproject.toml | grep '^version'
```

IMPORTANT

Continue only if `SHA` is not empty, the log shows the `Release X.Y.Z` commit, and the version line reads `version = "X.Y.Z"`.

4. Tag that commit and push the tag:

```
git tag -a "v$VERSION" "$SHA" -m "Release v$VERSION"
git push origin "v$VERSION"
```

5. Approve the upload. In GitHub → Actions, the `release` run builds and checks the tag against `pyproject.toml`. Its **Publish to PyPI** job then waits for you: **Review deployments** → **pypi** → **Approve and deploy**. The `docs` run deploys the site at the same time.

```
gh run list --workflow release.yml --limit 1
gh run list --workflow docs.yml --limit 1
```

6. Verify:

- ☐ <https://pypi.org/project/kpnn2/X.Y.Z/> shows the README with every image.
- ☐ <https://thomas-rauter.github.io/kpnn2/> opens `X.Y.Z` as `latest`.
- ☐ A clean install reports the version:

```
python -m venv /tmp/kpnn2-check
/tmp/kpnn2-check/bin/pip install "kpnn2==$VERSION"
/tmp/kpnn2-check/bin/python -c "import kpnn2; print(kpnn2.__version__)"
```

7. GitHub release

WARNING

Run this only after the tag is on GitHub. If the tag is missing, `gh` silently creates a new one from the tip of `main`.

In the same shell as section 6:

```
gh release create "v$VERSION" -t "kpnn2 v$VERSION" -F RELEASE_NOTES.md
```

Open the release page and check that the notes render and the changelog link works.

If the release run fails

- **Nothing reached PyPI** (for example a tag and version mismatch): fix the cause, remove the tag, and go back to section 5.

```
git push --delete origin "v$VERSION"
git tag -d "v$VERSION"
```

- **A broken `x.y.z` is on PyPI:** that version can never be uploaded again. Yank it on pypi.org (**Manage** → **Releases** → **Options** → **Yank**), fix the cause, and release `x.y.(z+1)` through this whole procedure.